Hash Tables

Design and Analysis of Algorithms

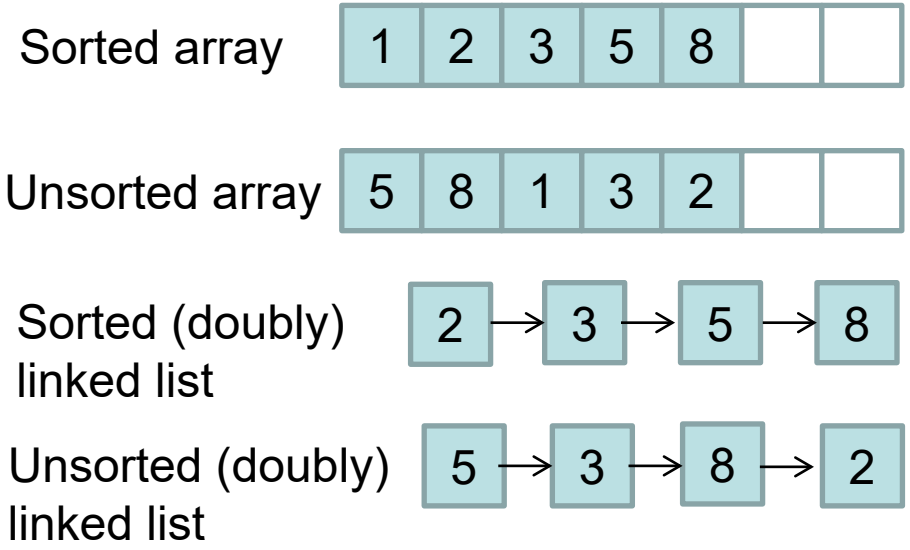
Brian C. Dean

School of Computing

Clemson University

Set / Map Data Structures

Insert	Remove	Find	
O(n)	O(n)	O(log n)	Sorted array
O(1)	O(1), post-find	O(n)	Unsorted array
O(1), post-find	O(1), post-find	O(n)	Sorted (doubly) linked list
O(1)	O(1), post-find	O(n)	Unsorted (doubly) linked list
O(log n)	O(log n)	O(log n)	Balanced BST, skip list, B-tree
O(1) amortized	O(1) amortized	O(1) expected	Universal hash tables



Hash Tables as Maps

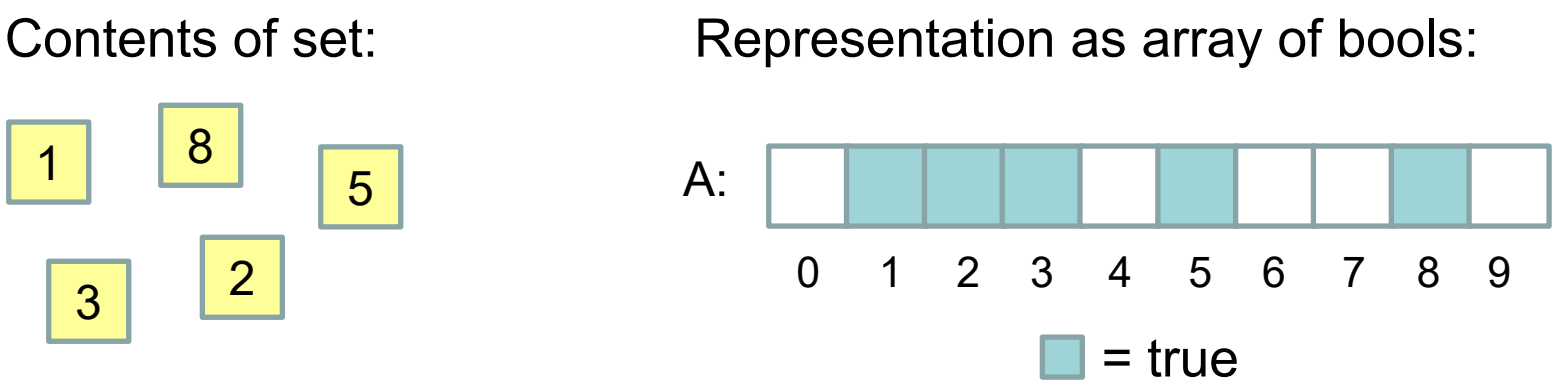
- Hash-based maps are supported in many languages (e.g., Perl, C++, Python, Javascript) using simple array notation:

```
grade["Brian"] = 100;
not_a_real_array[999999999999] = 3;
```

- Remember that even though these look like arrays, they are actually hash tables under the hood!
- Space-efficient and fast.

First Attempt: A Direct Access Table

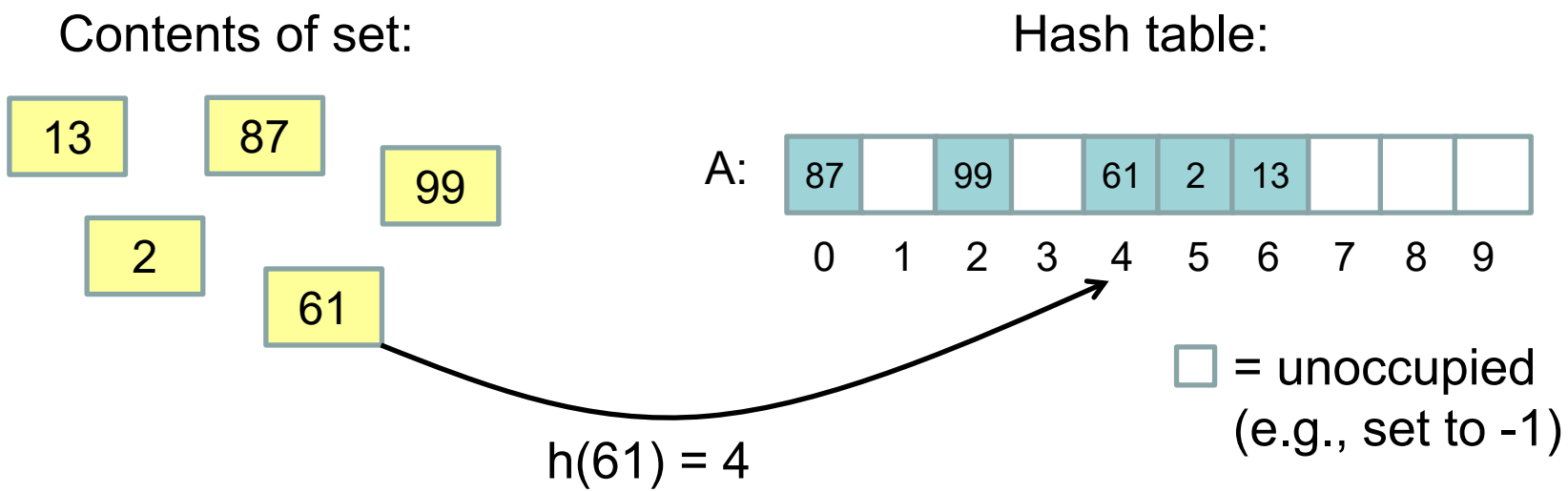
- Maintain a large array of bools
- Presence of key k in structure means A[k] is true.



- Insert, remove, and find all run in O(1) time!
- Serious drawback: **space usage** (and therefore also initialization time).

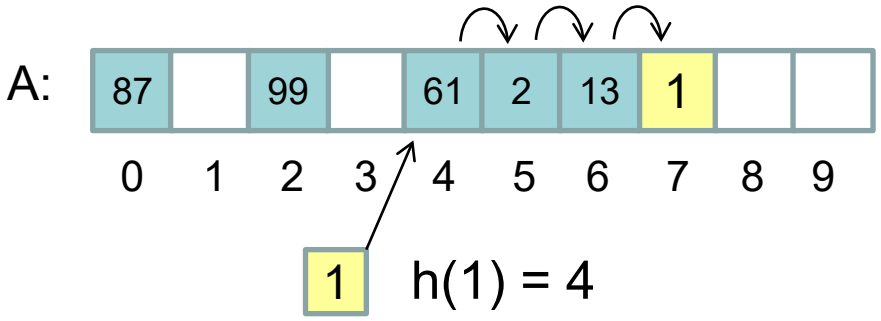
Hash Tables

- Store elements in an array.
- Key k stored in position h(k)
- h() is known as a "hash function" – it maps keys down to the range of indices in our array.
- Example: h(k) = (2971k + 101923) % 10.

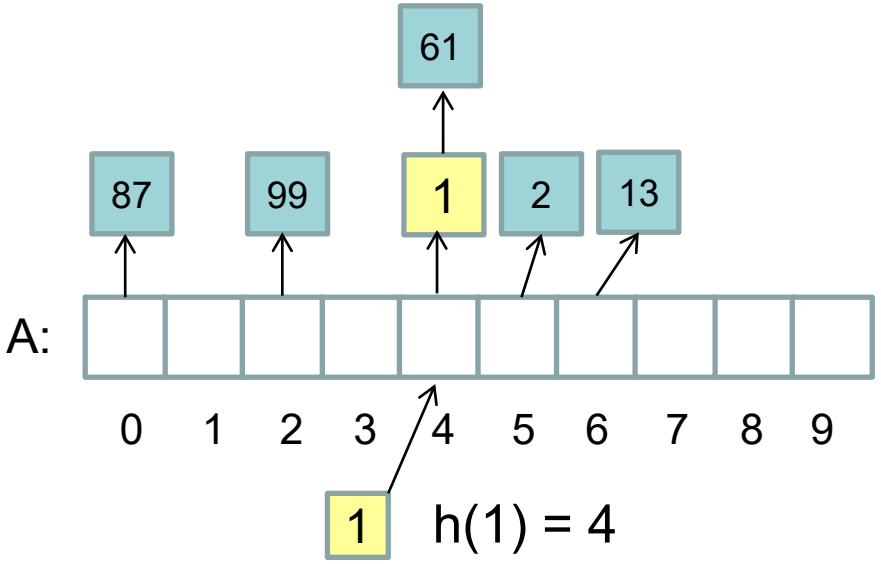


Collision Resolution

- Probing: store elements directly in table; careful when removing elements.
- Chaining: store linked list of colliding elements off each table cell (prepend, don't append).



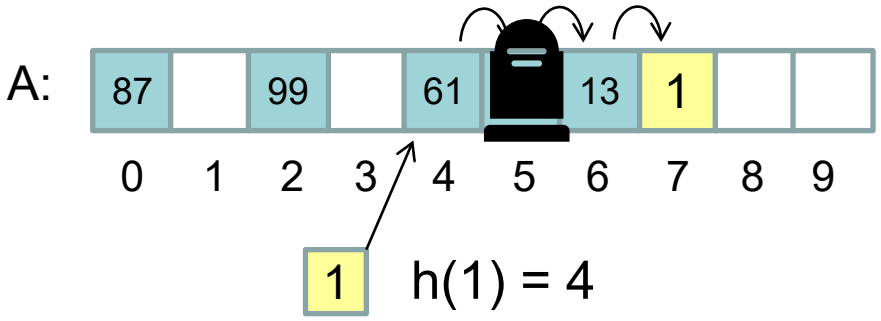
(Sequential) Probing



Chaining

Notes on Probing

- Sometimes more complicated probing patterns used to try and avoid clumps -- e.g., probe locations h(key, 1), h(key, 2), h(key, 3), etc. However, this may perform poorly due to caching.
- Careful when deleting elements!

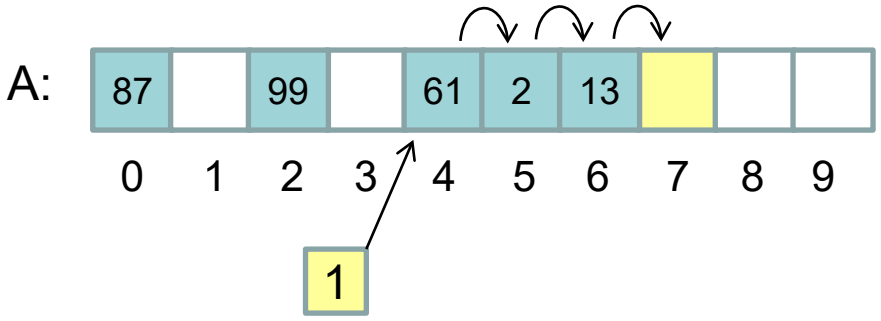


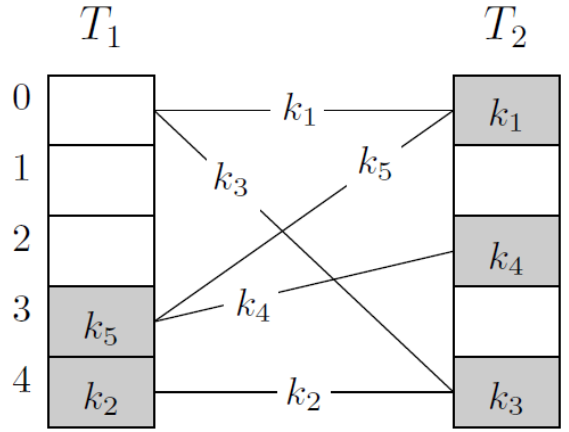
(Sequential) Probing

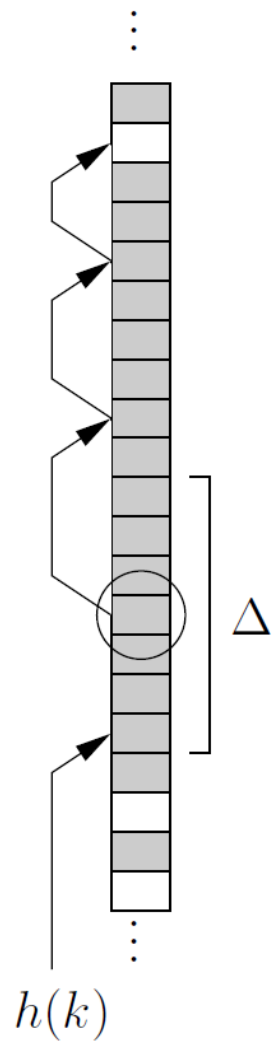
Chaining

Probing Strategies to Minimize Time for Finds

- Collision: challenge between a new element and the incumbent. One loses and moves on, the other stays.
- "Hopscotch" hashing: try to keep each key k within Δ of h(k).
- "Robin hood" hashing: favor the weary well-traveled key.
- "Cuckoo hashing": k found either at $T_1[h_1(k))$ or $T_2[h_2(k)]$ (may need to give up and rehash everything if not possible)

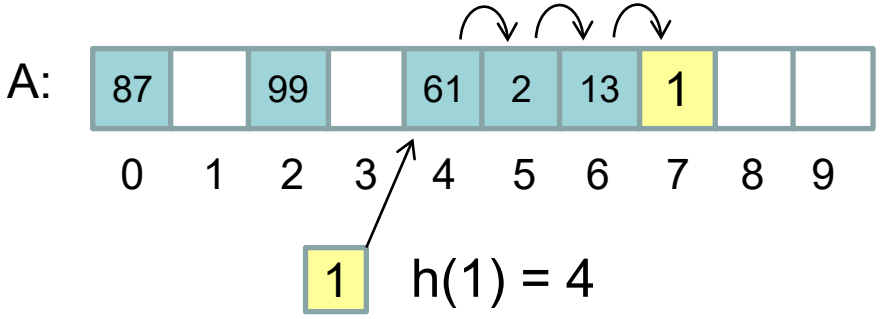




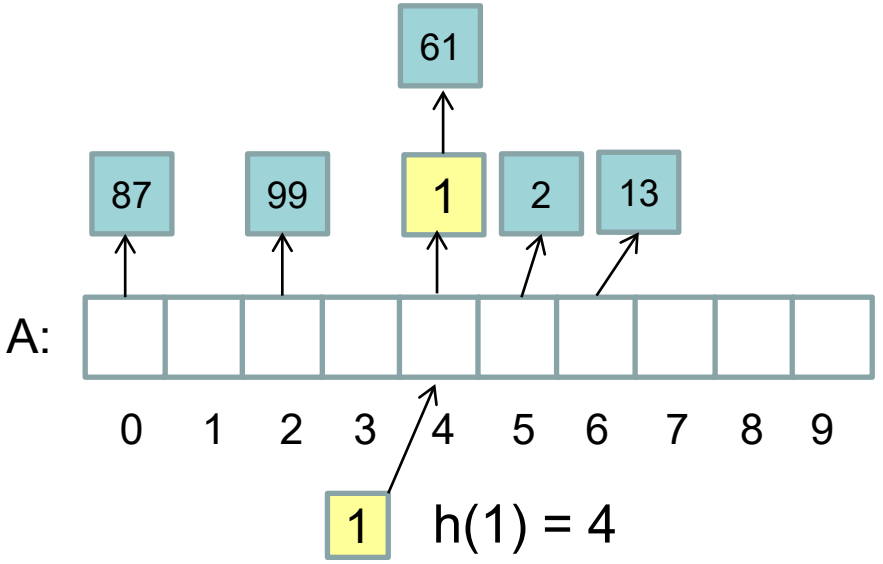


So a Hash Table Walks into a Bar…

- The bartender comes up after a while and says, “You ordered?”
- The hash table replies: “No.”



(Sequential) Probing



Chaining

Why “Hashing”…?

Hash (n) A dish of cooked meat cut into small pieces and recooked, usually with potatoes.

Hash (v) To jumble or mix up.

Choosing a Good Hash Function In Practice

- We want h() to behave somewhat “randomly”.
- Good Examples:
  - h(k) = k mod m.
  - h(k) = $\lfloor mk / C \rfloor$.
  - h(k) = (ak) mod m.
  - h(k) = (ak + b) mod m.

m = table size
C = max possible key value

a and b should be chosen in a somewhat “arbitrary” fashion (primes often used).

- Be sure to fully utilize “all the bits” in a key.
  - For example, h(k) = k mod 256 is a somewhat poor hash function if k is a 32-bit IP address.
- Be sure to use the entire hash table
  - For example, make sure h(k) isn’t always even.

Amortized Rebuilding

- How do we pick the initial size of our table?
- We ideally want to keep m = Θ(n), where m = table size and n = # of elements stored in table.
- To do this, double table size and re-hash when table becomes too full, and halve table size and re-hash when table becomes too empty.
- Since it takes only linear time to re-build the table (*), this makes insert and delete take + O(1) extra amortized time.

(*) possible and easy / obvious with chaining

(*) also possible, but maybe not immediately obvious, with sequential probing